Reporte: 2p ejercicio 20(pe)

En este programa se implementa el algoritmo Merge Sort utilizando la técnica de Ordenamiento Indirecto bajo el paradigma de Programación Estructurada (PE). En lugar de modificar la posición de los datos originales en memoria, el programa ordena un arreglo auxiliar de índices, permitiendo consultar la información de manera ordenada sin alterar el vector fuente.

Código Completo:

```
#include <iostream>
#include <vector>
#include <limits>

struct Registro {
    int clave;
    float valor;
};

void mergeIndirectoEnteros(const std::vector<int>& datos, std::vector<int>& indices, int
izq, int medio, int der) {
    int n1 = medio - izq + 1;
    int n2 = der - medio;
    std::vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = indices[izq + i];
    for (int j = 0; j < n2; j++) R[j] = indices[medio + 1 + j];
    int i = 0, j = 0, k = izq;
    while (i < n1 && j < n2) {
        if (datos[L[i]] <= datos[R[j]]) indices[k++] = L[i++];
        else indices[k++] = R[j++];
    }
    while (i < n1) indices[k++] = L[i++];
    while (j < n2) indices[k++] = R[j++];
}

void mergeSortIndirectoEnteros(const std::vector<int>& datos, std::vector<int>& indices,
int izq, int der) {
    if (izq < der) {
        int medio = izq + (der - izq) / 2;
        mergeSortIndirectoEnteros(datos, indices, izq, medio);
        mergeSortIndirectoEnteros(datos, indices, medio + 1, der);
        mergeIndirectoEnteros(datos, indices, izq, medio, der);
    }
}

void mergeIndirectoCaracteres(const std::vector<char>& datos, std::vector<int>& indices,
int izq, int medio, int der) {
    int n1 = medio - izq + 1;
    int n2 = der - medio;
    std::vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = indices[izq + i];
    for (int j = 0; j < n2; j++) R[j] = indices[medio + 1 + j];
    int i = 0, j = 0, k = izq;
    while (i < n1 && j < n2) {
        if (datos[L[i]] <= datos[R[j]]) indices[k++] = L[i++];
        else indices[k++] = R[j++];
```

```
    }
    while (i < n1) indices[k++] = L[i++];
    while (j < n2) indices[k++] = R[j++];
}

void mergeSortIndirectoCaracteres(const std::vector<char>& datos, std::vector<int>&
indices, int izq, int der) {
    if (izq < der) {
        int medio = izq + (der - izq) / 2;
        mergeSortIndirectoCaracteres(datos, indices, izq, medio);
        mergeSortIndirectoCaracteres(datos, indices, medio + 1, der);
        mergeIndirectoCaracteres(datos, indices, izq, medio, der);
    }
}

void mergeIndirectoRegistros(const std::vector<Registro>& datos, std::vector<int>&
indices, int izq, int medio, int der) {
    int n1 = medio - izq + 1;
    int n2 = der - medio;
    std::vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = indices[izq + i];
    for (int j = 0; j < n2; j++) R[j] = indices[medio + 1 + j];
    int i = 0, j = 0, k = izq;
    while (i < n1 && j < n2) {
        if (datos[L[i]].clave <= datos[R[j]].clave) indices[k++] = L[i++];
        else indices[k++] = R[j++];
    }
    while (i < n1) indices[k++] = L[i++];
    while (j < n2) indices[k++] = R[j++];
}

void mergeSortIndirectoRegistros(const std::vector<Registro>& datos, std::vector<int>&
indices, int izq, int der) {
    if (izq < der) {
        int medio = izq + (der - izq) / 2;
        mergeSortIndirectoRegistros(datos, indices, izq, medio);
        mergeSortIndirectoRegistros(datos, indices, medio + 1, der);
        mergeIndirectoRegistros(datos, indices, izq, medio, der);
    }
}

int main() {
    int opcion = 0;
    do {
        std::cout << "\n================================================\n";
        std::cout << "   MERGE SORT INDIRECTO (PE) - DIVERSOS TIPOS DE DATO\n";
        std::cout << "================================================\n";
        std::cout << "1. Ordenamiento indirecto de Enteros\n";
        std::cout << "2. Ordenamiento indirecto de Caracteres\n";
        std::cout << "3. Ordenamiento indirecto de Nuevo Tipo de Dato (Registro)\n";
        std::cout << "4. Salir\n";
        std::cout << "Seleccione una opción: ";
        if (!(std::cin >> opcion)) break;

        if (opcion == 1) {
            int n;
            std::cout << "Cantidad de enteros: ";
            std::cin >> n;
```

```
            if (n <= 0) continue;
            std::vector<int> datos(n);
            std::vector<int> indices(n);
            for (int i = 0; i < n; i++) {
                std::cout << "Entero [" << i << "]: ";
                std::cin >> datos[i];
                indices[i] = i;
            }
            mergeSortIndirectoEnteros(datos, indices, 0, n - 1);
            std::cout << "\n--- DATOS ORIGINALES (SIN MOVER) ---\n[ ";
            for (int i = 0; i < n; i++) std::cout << datos[i] << " ";
            std::cout << "]\n";
            std::cout << "--- ÍNDICES ORDENADOS ---\n[ ";
            for (int i = 0; i < n; i++) std::cout << indices[i] << " ";
            std::cout << "]\n";
            std::cout << "--- DATOS ACCEDIDOS DE FORMA INDIRECTA (ORDENADOS) ---\n[ ";
            for (int i = 0; i < n; i++) std::cout << datos[indices[i]] << " ";
            std::cout << "]\n";
        }
        else if (opcion == 2) {
            int n;
            std::cout << "Cantidad de caracteres: ";
            std::cin >> n;
            if (n <= 0) continue;
            std::vector<char> datos(n);
            std::vector<int> indices(n);
            for (int i = 0; i < n; i++) {
                std::cout << "Caracter [" << i << "]: ";
                std::cin >> datos[i];
                indices[i] = i;
            }
            mergeSortIndirectoCaracteres(datos, indices, 0, n - 1);
            std::cout << "\n--- DATOS ORIGINALES (SIN MOVER) ---\n[ ";
            for (int i = 0; i < n; i++) std::cout << datos[i] << " ";
            std::cout << "]\n";
            std::cout << "--- ÍNDICES ORDENADOS ---\n[ ";
            for (int i = 0; i < n; i++) std::cout << indices[i] << " ";
            std::cout << "]\n";
            std::cout << "--- DATOS ACCEDIDOS DE FORMA INDIRECTA (ORDENADOS) ---\n[ ";
            for (int i = 0; i < n; i++) std::cout << datos[indices[i]] << " ";
            std::cout << "]\n";
        }
        else if (opcion == 3) {
            int n;
            std::cout << "Cantidad de registros: ";
            std::cin >> n;
            if (n <= 0) continue;
            std::vector<Registro> datos(n);
            std::vector<int> indices(n);
            for (int i = 0; i < n; i++) {
                std::cout << "\nRegistro " << i << ":\n";
                std::cout << "  Clave (entero): ";
                std::cin >> datos[i].clave;
                std::cout << "  Valor (float): ";
                std::cin >> datos[i].valor;
                indices[i] = i;
            }
            mergeSortIndirectoRegistros(datos, indices, 0, n - 1);
```

```
            std::cout << "\n--- REGISTROS ACCEDIDOS INDIRECTAMENTE (ORDENADOS POR CLAVE)
---\n";
            for (int i = 0; i < n; i++) {
                int idx = indices[i];
                std::cout << "Posición ordenada " << i << " -> Índice original " << idx
                          << " | Clave: " << datos[idx].clave << " | Valor: " <<
datos[idx].valor << "\n";
            }
        }
        else if (opcion == 4) {
            std::cout << "Saliendo...\n";
        }
        else {
            std::cout << "Opción no válida.\n";
        }
    } while (opcion != 4);
    return 0;
}
```

Explicación del Código:

1. Estructura de Datos Personalizada (struct Registro)

Define una estructura compuesta con los campos clave (entero) y valor (float). Permite manejar registros personalizados ordenados indirectamente a través de su clave.

2. Lógica de Ordenamiento Indirecto (Funciones mergeIndirecto...)

A diferencia del ordenamiento directo, las funciones de mezcla comparan los elementos originales consultándolos mediante los índices guardados en los subvectores L y R (por ejemplo, datos[L[i]] <= datos[R[j]]), pero los movimientos y reordenamientos se realizan exclusivamente sobre el arreglo de índices.

3. Funciones Auxiliares 'Divide y Vencerás' (Funciones mergeSortIndirecto...)

Dividen recursivamente el vector de índices a la mitad hasta fragmentarlo en elementos individuales, para posteriormente combinarlos ordenadamente mediante la función de mezcla correspondiente.

4. Menú Interactivo y Visualización (main)

Implementa un ciclo do-while que permite al usuario elegir entre enteros, caracteres o registros. Inicializa el arreglo de índices secuencialmente (indices[i] = i) y al finalizar imprime los datos originales intactos, los índices ordenados y el resultado final consultando los datos de forma indirecta (datos[indices[i]]).